

Informe del Proyecto 2 - PAHM

Alexandra Alfaro Elizondo, Kendall Madrigal Campos y Bryan Molina Lopez
Escuela de Ingeniería Electrónica, Instituto Tecnológico de Costa Rica (ITCR), 30101 Cartago, Costa Rica
{alexvaneska, kendallmacampos2941, bryanmolinalo772}@estudiantec.cr

20 de junio de 2026

1. Resumen

Este repositorio contiene la línea de software e infraestructura de simulación para el Proyecto 2. El objetivo del proyecto es modelar las perturbaciones de viento latentes no observadas mediante celdas recurrentes (aprendizaje de secuencias) y diseñar una política de control robusta utilizando Aprendizaje por Refuerzo (RL) sobre un entorno Gymnasium acoplado a un modelo físico híbrido (Caja Gris).

2. Hardware y Entorno de Ejecución

Completar esta sección con las características del equipo usado para las corridas finales.

Cuadro 1: Características del hardware y entorno por integrante.

Campo	Bryan	Alexandra	Kendall
Sistema operativo	CachyOS	Ubuntu 24.04.4 LTS	Ubuntu 24.04.4 LTS
CPU	Core i5	AMD Ryzen 7 PRO 7840HS	AMD Ryzen 7 7735HS
RAM	16 GB	32 GB	16 GB
GPU	NVIDIA RTX 3050	-	Radeon RX 7700S
VRAM	6 GB	-	8 GB
Versión de Python	3.12.13	3.12.13	3.12.13
Versión de PyTorch	2.3.1+cu121	2.12.0+cpu	2.3.1+cu121
Versión de Stable Baselines3	No instalado	No instalado	2.8.0
Versión de CUDA/cuDNN	CUDA 12.1 & cuDNN 8902	-	ROCm
Entorno o gestor	uv	uv	uv

3. NFR-7 - Tiempos de Ejecución

Registrar los tiempos finales usando la misma máquina declarada en la sección anterior. Si se repite una corrida, reportar media y desviación estándar o indicar explícitamente que se hizo una sola medición.

3.1. Comandos sugeridos para medir

Usar `/usr/bin/time -v` cuando se quiera capturar tiempo de pared y memoria máxima residente:

Cuadro 2: Registro de procesos y tiempos de ejecución.

Proceso	Comando script	Configuración relevante	Tiempo total	Unidad	Artefacto de salida	Observaciones
Entrenamiento estimador	python pahm_model/train_estimator.py	epochs=40, tau_max=2.0, ...	~44.7	min	checkpoints/estimator...	Tiempo promedio por época ≈ 67 s. Checkpoint época 40 seleccionado por convergencia en W&B.
Inferencia estimador	python pahm_model/export_tau.w.py	checkpoint: época 40	46.13	s	outputs/tau.w/	38 archivos .numpy procesados (train+val+test). Coherente con tau_max=2.0.
Entrenamiento Stage 2	.venv/bin/python -m pahm_stage2...	GMM + BIC			artifacts/stage2/gmm_wind_model.pkl	
Entrenamiento RL naive	... train_rl.py --mode naive	wind_enabled=false			artifacts/stage3/pahm_ppo_naive.zip	
Entrenamiento RL robust	... train_rl.py --mode robust	wind_source=stage2.sampler			artifacts/stage3/pahm_ppo_robust.zip	
Evaluación controladores	... evaluate_controllers.py	patrones: gust, turbulent			artifacts/stage3/evaluation/	

```

/usr/bin/time -v .venv/bin/python train_rl.py --config gym_wrapper/config.json --mode naive
/usr/bin/time -v .venv/bin/python train_rl.py --config gym_wrapper/config.json --mode robust
/usr/bin/time -v .venv/bin/python evaluate_controllers.py --config configs/stage3_config.json

```

Para scripts ejecutados con python del sistema en vez del entorno virtual, mantener consistencia y registrar cuál intérprete se usó.

4. Etapa 1 - Estimador de Perturbaciones

Arquitectura del estimador.

Red recurrente GRU (WindSequenceEstimator, pahm_model/sequence_estimator.py) con hidden_dim=64, num_layers=2, ventana de entrada sequence_length=10 pasos con 4 features por paso: $(\sin \theta, \cos \theta, d\theta/dt, u)$. Cumple CON-2: no recibe el residuo $r(t) = \theta_{obs} - \theta_{ODE}$ como entrada directa. La salida escalar $\hat{\tau}_w(t)$ se acota con $\tanh(x) \cdot \tau_{max}$ ($\tau_{max} = 2,0$) para evitar que el entrenamiento converja a torques no físicos.

Integración con la física. El torque estimado se inyecta de forma aditiva dentro del paso RK4 de PAHMPhysicsCell (modelo del profesor, pahm_model/pahm_fast.py), llamando directamente a physics_model.cell(state, control, tau=tau.w) desde rk4_integrator.py (TaylorWindIntegrator). Los parámetros físicos (α, β, γ) permanecen congelados durante todo el entrenamiento (CON-1), verificado mediante requires_grad=False en todos los parámetros de physics_model.

Función de pérdida.

Tres términos según la Ecuación 13 del enunciado (pahm_model/custom_loss.py): reconstrucción ($F.mse_{loss}$), parsimonia ($\lambda_1 \cdot \text{mean}(\tau_w^2)$ sobre toda la ventana) y suavidad temporal ($\lambda_2 \cdot \text{mean}(\Delta \tau_w^2)$). Coeficientes finales: $\lambda_1 = 0,05$, $\lambda_2 = 0,1$, ambos configurables externamente

en `gym_wrapper/config.json`.

Checkpoint usado. `checkpoints/estimator_checkpoint_epoch_40.pth`, seleccionado por convergencia observada en las curvas de W&B (estabilización de `loss_total`, `loss_parsimony` y `loss_smoothness` desde aproximadamente la época 18-20), no por agotar las 40 épocas configuradas --- no hay un número de épocas obligatorio impuesto por el enunciado.

Métrica de comparación en lazo abierto (FR-7). Evaluación sobre el conjunto de prueba retenido (`test/test_verification.py::test_open_loop_baseline_comparison_fr7`), comparando ODE pura vs. ODE + estimador GRU:

Configuración	MSE (lazo abierto, test set)
ODE pura (sin viento)	0.00000455
ODE + estimador GRU	0.00000437
Mejora relativa	3.90 %

El estimador iguala/supera al baseline tal como exige el enunciado (sección 5.2.3 de `proy2.pdf`). La mejora es modesta en magnitud absoluta pero estadísticamente consistente sobre trayectorias no vistas en entrenamiento.

Verificación con datos sintéticos (NFR-6f). El estimador recupera un torque de viento constante conocido ($\tau_w = 0,5$) inyectado en una trayectoria sintética generada por la misma física, con error final $< 0,25$ tras 150 iteraciones de optimización con `lr=0.03` (`test/test_verification.py::test_synthetic_wind_recovery_nfr6f`).

Tiempo de entrenamiento. ≈ 44.7 minutos (`67 s/época promedio  $\times$  40 épocas`) en la GPU declarada en la sección 2. Ver detalle en la tabla de la sección 3.

Tiempo de inferencia/exportación de $\tau_w(t)$. `46.13 s` para procesar las 38 trayectorias completas del dataset (`train + val + test`) y generar los `.npy` consumidos por el Grupo 2, más el checkpoint copiado como `outputs/tau_w/estimator_wind.pth` (Hito I-1).

Rango de $\tau_w(t)$ estimado. Sobre las 38 trayectorias exportadas: `mín = -2,0000`, `máx = 2,0000`, `mean = -0,5205`, `std = 1,5810` --- acotado por diseño al límite $\tau_{max} = 2,0$.

5. Etapa 2 - Representación No Superficial

Nota sobre dos implementaciones en el repositorio. Existen dos carpetas con funcionalidad de Etapa 2: `etapa2_unsupervised/` (un autoencoder simple sobre ventanas de $\tau_w(t)$, optimizando solo MSE de reconstrucción) y `pahm_stage2/` (un modelo de mezcla gaussiana --- GMM --- con selección de complejidad por BIC, validación sintético-real con ARI/NMI, y `WindSampler` como consumidor formal hacia Etapa 3). Según el propio `README.md` de `etapa2_unsupervised/`, esa carpeta es un prototipo histórico descartado y se conserva solo para trazabilidad; la entrega oficial de Etapa 2 es `pahm_stage2/`. Lo que sigue documenta la versión vigente (GMM + BIC).

Features extraídas de $\tau_w(t)$ (`pahm_stage2/feature_extractor.py`): `mean`, `std`, `max_abs`, `skewness`, `smoothness` (media del cuadrado de la primera diferencia) y `energy` (media de los valores al cuadrado). Las señales de entrada tienen longitud variable; el extractor las reduce a un vector de tamaño fijo por trayectoria antes de pasarlas al GMM.

Criterio BIC para selección de componentes (`pahm_stage2/unsupervised_model.py`, clase `GMMWindModel`): se entrenan modelos `GaussianMixture` (covarianza completa, `n_init=5`) para un rango de componentes configurable, y se selecciona el número de componentes

con el BIC más bajo. Sobre los datos sintéticos generados con cuatro patrones de viento conocidos (calm, gust, bias, turbulent), los puntajes BIC obtenidos fueron:

Componentes	BIC
2	-3937.42
3	-5878.80
4	-7243.61
5	-7236.33
6	-7150.94

El BIC mínimo se alcanza en 4 componentes, exactamente el número de patrones sintéticos inyectados --- el criterio recupera la estructura correcta sin haber recibido las etiquetas. La curva muestra una caída pronunciada hasta 4 componentes y luego se aplana/empeora levemente, confirmando que no hay sobreajuste por agregar componentes adicionales.

Métricas ARI/NMI (pahm_stage2/validator.py, validación sintético-real sobre los cuatro patrones conocidos):

Métrica	Valor
ARI (Adjusted Rand Index)	1.0
NMI (Normalized Mutual Information)	1.0
Componentes seleccionados	4

Ambas métricas en 1.0 indican una recuperación perfecta de la partición verdadera sobre los datos sintéticos --- condición necesaria, según el enunciado, antes de aplicar el mismo procedimiento de forma exploratoria sobre los datos reales de $\tau_w(t)$ (sin verdad de campo disponible).

Checkpoint GMM generado: persistido vía `GMMWindModel.save()` en la ruta configurada en `configs/stage2_config.json` (`unsupervised.checkpoint_path`). El mismo checkpoint es consumido por `WindSampler.from_checkpoint()` (`pahm_stage2/wind_sampler.py`) para muestrear vectores de perturbación latente como fuente para Etapa 3 (FR-11).

6. Etapa 3 - Control Robusto RL

Algoritmo y configuración base (`train_rl.py`, `DEFAULT_RL_TRAINING`): algoritmo PPO de Stable Baselines3 (también soporta A2C y SAC por configuración), `total_timesteps` y demás hiperparámetros leídos desde `rl_training` en el config externo. El entorno se construye sin renderizado (`render_mode=None`) mediante `LearnedPAHMODE`.

Diferencia entre entrenamiento naive y robust (`DEFAULT_EXPERIMENTS` en `train_rl.py`):

Modo	wind_enabled	wind.source configurado por defecto
naive	False	— (sin perturbaciones durante el entrenamiento)
robust	True	stage2_sampler

El diseño original del modo robust apunta a usar el `WindSampler` del GMM de Etapa 2 (`wind.source="stage2_sampler"`) como fuente de perturbaciones durante el entrenamiento, cumpliendo FR-11 (la representación aprendida alimenta el controlador robusto). Cada modo es entrenado de forma independiente vía `train_from_config(config_path, mode=mode)`,

con su propio nombre de modelo (pahm_ppo_naive / pahm_ppo_robust) y su propio registro en W&B (proyecto pahm-rl-stage3).

Evaluación cuantitativa --- fuente de viento real usada. El script `evaluate_controllers.py` (`make_evaluation_env`) construye el entorno de evaluación pasando únicamente `wind_pattern` (de `unseen_wind_patterns`: `gust`, `turbulent`) y no propaga `wind_source`. Por diseño, esto hace que la evaluación final de ambos controladores corra sobre perturbaciones de `WindProcess`, no del `WindSampler` de Etapa 2 --- consistente con la columna `wind source=wind_process` observada en `controller_metrics.csv` para todos los episodios de naive y robust. Es decir: la diferencia entre ambos controladores en la evaluación final proviene de cómo fue entrenado cada uno (sin perturbaciones vs. con perturbaciones activas), no de la fuente de viento usada al evaluarlos.

Observación y referencia. El entorno `LearnedPAHMODE` expone `theta_ref` configurable (por `config`, `constructor`, `set_theta_ref()` o `reset(options={"theta_ref": ...})`) y reporta `theta_ref`, `tracking_error` y `abs_tracking_error` en `info`. La recompensa de seguimiento combina error cuadrático, velocidad angular y esfuerzo de acción con pesos configurables (`reward.tracking_error_weight`, `reward.velocity_weight`, `reward.control_weight`).

7. Comparación de Controladores

Usar los resultados generados por:

```
.venv/bin/python evaluate_controllers.py --config configs/stage3_config.json
```

Artefactos esperados:

- `artifacts/stage3/evaluation/controller_metrics.json`
- `artifacts/stage3/evaluation/controller_metrics.csv`
- `artifacts/stage3/evaluation/informe.md`

Copiar aquí la tabla resumen naive vs robust y la conclusión generada, revisada por el equipo humano.

8. Weights & Biases

Todas las corridas del proyecto (Etapas 0, 1, 2 y 3) se registran bajo un único workspace de equipo en Weights & Biases:

Workspace: `wandb.ai/aprendi1s26`

Dentro de ese workspace existen tres proyectos, uno por etapa grupal del proyecto (la Etapa 0, individual por integrante, también es accesible desde el mismo workspace, cada integrante registró sus corridas de DQN ahí):

Proyecto W&B	Etapla correspondiente
etapa-1	Etapla 1 — Estimador de Perturbaciones de Viento (Grupo 1)
etapa-2-unsupervised	Etapla 2 — Representación No Supervisada (Grupo 2)
pahm-rl-stage3	Etapla 3 — Control Robusto RL (Grupo 2)